A

Project Report on

Process Scheduling

Submitted for partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+, ISO 9001-2008 Certified.

Dhulapally, Secunderabad-500 100

JUNE- 2026

St. MARTIN'S ENGINEERING COLLEGE

UGC AUTONOMOUS

Affiliated to JNTUH, Approved by AICTE Accredited by NBA & NAAC A+, ISO 9001-2008 Certified.

Dhulapally, Secunderabad-500 100

CERTIFICATE

This is to certify that the project titled “Process Scheduling” is being submitted by Darshil Mishra (24K81A05L7) in fulfillment of the requirement for the award of degree of BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING is recorded of bonafide work carried out by them. The result embodied in this report have been verified and found satisfactory.

Signature of Guide Mr. M. Dileep Kumar Assistant Professor Department of CSE

Signature of HOD

Mr. D. Krishna Kishore Associate professor & HOD Department of CSE

Internal Examiner External Examiner

St. MARTIN'S ENGINEERING COLLEGE

UG C AUTONOMOUS

Affiliated to JNTUH, Approved by AICTE Accredited by NBA & NAAC A+, ISO 9001-2008 Certified.

Dhulapally,Secunderabad-500 100

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I, the student of “Bachelor of Technology in Department of Computer Science & Engineering”, session: 2024 - 2028, St. Martin’s Engineering College, Dhulapally, Kompally, Secunderabad, hereby declare that the work presented in this project work entitled “Process Scheduling” is the outcome of my own bonafide work and is correct to the best of my knowledge and this work has been undertaken taking care of Engineering Ethics. This result embodied in this project report has not been submitted in any university for award of any degree.

Darshil Mishra(24K81A05L7)

Acknowledgement

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragement and guidance have crowded our efforts with success.

First and foremost, I would like to express our deep sense of gratitude and indebtedness to our College Management for their kind support and permission to use the facilities available in the Institute.

I especially would like to express our deep sense of gratitude and indebtedness to Prof. Dr. K. RAVINDRA, Director & Principal, St. Martin's Engineering collage Dhulapally, for permitting us to undertake this project.

I am also thankful to, MR. D. KRISHNA KISHORE Head of the Department, Computer science and engineering, St. Martin's Engineering College, Dhulapally, Secunderabad, for his support and guidance throughout our project as well as Project Coordinator Mr. M. Dileep Kumar, Assistant Professor, Department of Computer science and Engineering for his valuable support.

I would like to express my sincere gratitude and indebtedness to my project supervisor Mr. M. Dileep Kumar Assistant Professor, Computer Science and Engineering, St. Martin's Engineering College, Dhulapally, for his support and guidance throughout our project.

Finally, I express thanks to all those who have helped me successfully completing this project. Furthermore, I would like to thank my family and friends for their moral support and encouragement. I express thanks to all those who have helped me in successfully completing the project.

Darshil Mishra (24K81A05L7)

CONTENTS

ABSTRACT

This project, Process Scheduling Simulator, presents a comprehensive and interactive academic simulation of CPU process scheduling within an operating system environment. The simulator enables users to explore and compare the performance of various CPU scheduling algorithms, including First Come First Serve (FCFS), Shortest Job First (SJF), Priority Scheduling, and Round Robin Scheduling.The Process Scheduling Simulator offers a user-friendly interface for inputting process-related data, such as Process ID, Arrival Time, Burst Time, Priority Level, and Time Quantum. Users can select from the supported scheduling algorithms and observe the execution order of processes based on each algorithm's logic. The simulator calculates and displays key scheduling metrics, including Waiting Time, Turnaround Time, Completion Time, and Response Time.By utilizing the Process Scheduling Simulator, users can gain a deeper understanding of CPU scheduling concepts and process management principles in operating systems. The simulator's ability to store process execution details and results enables users to conduct repeated analysis and comparison, facilitating a comprehensive evaluation of the strengths and limitations of different scheduling algorithms. This project provides a practical tool for educational purposes, allowing users to simulate and evaluate CPU scheduling techniques, ultimately enhancing their conceptual understanding of process execution and scheduling performance.

INTRODUCTION

The development of efficient operating systems relies heavily on the effective management of CPU resources among multiple processes. Process scheduling, a fundamental concept in operating systems, plays a crucial role in ensuring that system resources are utilized optimally. As the complexity of modern operating systems continues to grow, the need for intuitive and efficient process scheduling algorithms has become increasingly pressing. This project aims to address this need by developing a comprehensive Process Scheduling Simulator, designed to provide users with a deeper understanding of CPU scheduling concepts and their practical implementation.

The development of the Process Scheduling Simulator is motivated by the need for a practical and interactive tool that can aid in the understanding of process management concepts. By allowing users to experiment with different scheduling algorithms and observe their effects on CPU utilization, process completion speed, and overall scheduling performance, the simulator provides a valuable educational resource for students and professionals alike. Furthermore, the system's ability to store process execution details and results enables users to conduct repeated analysis and comparison, facilitating a deeper understanding of the complexities involved in process scheduling.

Objectives :

The primary objectives of this project are outlined below:

Develop a CPU process scheduling simulator for operating systems.

Demonstrate the execution of processes using different scheduling algorithms.

Compare the performance of various scheduling strategies and techniques.

Improve understanding of process management concepts in operating systems.

Evaluate the strengths and limitations of different scheduling algorithms.

Scope :

The scope of the Process Scheduling Simulator project encompasses the following key aspects:

The simulator will support five CPU scheduling algorithms.

The system will accept user-defined process-related inputs.

The project will calculate and display scheduling metrics.

The simulator will generate a Gantt Chart representation.

The system will store process execution details and results.

LITERATURE SURVEY

The literature survey for this project involved a comprehensive review of existing works related to CPU process scheduling in operating systems. The survey focused on identifying the strengths and limitations of various scheduling algorithms, their implementation, and their impact on

system performance. Several research papers and articles were analyzed to gain insights into the scheduling behaviors of different algorithms, including First Come First Serve (FCFS), Shortest Job First (SJF), Priority Scheduling, and Round Robin Scheduling.

Key findings from the literature survey are summarized below:

Several studies have shown that Round Robin Scheduling outperforms other algorithms in terms of CPU utilization.

Priority Scheduling has been found to be effective in reducing waiting times for high-priority processes.

SJF has been shown to achieve optimal performance in minimizing turnaround times for processes with varying burst times.

FCFS has been criticized for its poor performance in scenarios with high variability in process arrival times.

Research has also highlighted the importance of considering factors like process priority, burst time, and time quantum in scheduling algorithm design.

In addition to these findings, the literature survey also identified several gaps in existing research related to process scheduling. These gaps include the need for more comprehensive performance evaluation frameworks, the development of more efficient scheduling algorithms for real-time systems, and the investigation of the impact of scheduling algorithms on power consumption in embedded systems. Addressing these gaps is essential for improving the efficiency and effectiveness of CPU process scheduling in modern operating systems.

SYSTEM ANALYSIS

Existing System

The existing system for process scheduling is a traditional, manual approach that relies heavily on human intervention. This approach often results in inefficient CPU utilization, increased waiting times, and poor process completion speeds. It typically involves manually allocating tasks to available CPU resources, tracking process execution, and calculating scheduling metrics.

In the existing system, process scheduling is often performed using simple, ad-hoc methods that do not account for the complexities of modern operating systems. As a result, these methods often fail to maximize efficiency and minimize waiting times. Furthermore, the lack of a standardized framework for scheduling process execution makes it challenging to compare the performance of different algorithms.

Some key limitations of the existing system include:

Manual process scheduling is time-consuming and prone to errors.

Inefficient CPU utilization leads to increased waiting times and poor process completion speeds.

Lack of a standardized framework for scheduling process execution.

Limited ability to compare the performance of different scheduling algorithms.

No provision for storing process execution details and results for repeated analysis and comparison.

Proposed System

The proposed system aims to maintain the existing functionality of the Process Scheduling Simulator while incorporating enhancements to improve usability and maintainability. Key features of the proposed system include:

User-friendly interface for efficient input and configuration.

Enhanced algorithm selection and visualization capabilities.

Real-time scheduling metrics and Gantt Chart updates.

Support for multiple process execution scenarios and comparisons.

Data storage and retrieval mechanism for repeated analysis.

SYSTEM REQUIREMENTS

The system should be configured to run on a 64-bit operating system to ensure compatibility with the project's dependencies and libraries. It is also recommended to use a 4K or higher resolution display for optimal visualization of the Gantt chart.

Hardware Requirements

Intel Core i5 or equivalent processor.

8 GB RAM or more.

256 GB hard disk space or more.

Graphics card with 2 GB memory or more.

Operating System: Windows 10 or Linux Ubuntu 20.04.

Software Requirements

Java Development Kit (JDK) version 8 or above.

Eclipse IDE or similar integrated development environment.

Graphviz library for generating Gantt charts.

MySQL database management system for storing execution results.

Python scripting environment for data analysis and visualization.

ALGORITHM

Step 1: Initialize Process List.

The simulator accepts process-related inputs such as Process ID, Arrival Time, Burst Time, Priority Level, and Time Quantum.

Step 2: Validate Input Parameters.

Input parameters are validated for correctness and consistency to ensure accurate simulation results.

Step 3: Sort Process List (SJF).

For Shortest Job First (SJF) scheduling, the process list is sorted in ascending order of Burst Time to prioritize the shortest process.

Step 4: Apply Scheduling Algorithm.

The selected scheduling algorithm (FCFS, SJF, Priority, or Round Robin) is applied to the sorted process list to determine the execution order.

Step 5: Calculate Scheduling Metrics.

After execution, the simulator calculates scheduling metrics such as Waiting Time, Turnaround Time, Completion Time, and Response Time for each process.

Step 6: Generate Gantt Chart.

A Gantt Chart representation is generated to visually display the process execution timeline and CPU allocation order, providing a clear comparison of the selected scheduling algorithm's performance.

.

SYSTEM IMPLEMENTATION

The system implementation of the Process Scheduling Simulator was carried out using Java programming language. Java's object-oriented design and platform independence made it an ideal choice for building a simulator that can be seamlessly integrated with various operating systems.

The system is divided into three primary modules: Input Processing, Scheduling Algorithm Execution, and Result Calculation. The Input Processing module is responsible for accepting user inputs, validating process-related data, and preparing it for scheduling. The Scheduling Algorithm Execution module implements the CPU scheduling algorithms, including FCFS, SJF, Priority Scheduling, and Round Robin Scheduling. This module applies the selected algorithm to the input processes, determining their execution order and calculating relevant scheduling metrics.

The Result Calculation module computes the scheduling metrics, such as Waiting Time, Turnaround Time, Completion Time, and Response Time, based on the execution order and process characteristics. This module also generates a Gantt Chart representation of the process execution timeline and CPU allocation order. The simulator's graphical user interface (GUI) displays the calculated results and Gantt Chart, allowing users to visually analyze and compare the performance of different scheduling algorithms.

The system architecture is designed to facilitate extensibility and scalability. The scheduling algorithm implementation is decoupled from the GUI and input processing, enabling the easy addition of new algorithms or modifications to existing ones. This modular design ensures that the simulator remains flexible and adaptable to changing user requirements and evolving CPU scheduling strategies.

The simulator's data storage component allows for the saving of process execution details and results for repeated analysis and comparison. This feature enables users to track the performance of different algorithms over time, facilitating a deeper understanding of their strengths and limitations.

OUTPUT

CONCLUSION

The Process Scheduling Simulator project has successfully achieved its objectives, providing a comprehensive understanding of CPU scheduling concepts and performance evaluation. The simulator's ability to apply various scheduling algorithms and calculate relevant metrics has enabled users to analyze the strengths and limitations of each technique. This has contributed to a deeper conceptual understanding of process management and the allocation of CPU resources in operating systems.

Through the implementation of the simulator, it has become evident that different scheduling algorithms possess unique characteristics and advantages. The comparison of execution behavior, CPU utilization, and process completion speed under various algorithms has provided valuable insights into the design and optimization of scheduling systems. The ability to store and analyze process execution details has further enhanced the simulator's educational value, allowing users to identify trends and patterns in scheduling performance.

In conclusion, the Process Scheduling Simulator has proven to be a valuable tool for understanding CPU scheduling concepts and evaluating the performance of different scheduling algorithms. Its practical demonstration of scheduling logic and metrics has improved conceptual understanding and facilitated analysis of scheduling strategies. As an educational tool, the simulator is well-suited for use in operating systems courses and has the potential to contribute to the development of more efficient and effective scheduling systems in future operating system designs.

FUTURE ENHANCEMENTS

Implement Support for Multi-Level Feedback Queue (MLFQ) scheduling algorithm.

Introduce a graphical user interface (GUI) for improved user experience.

Develop a feature to analyze and compare scheduling performance metrics.

Enhance the system to handle process preemption and scheduling context switches.

Incorporate support for other CPU scheduling algorithms, such as Rate Monotonic Scheduling (RMS).

Introduce a feature to simulate process synchronization and communication techniques.

Develop a module to visualize process execution behavior using animation.

Improve the system's ability to handle large-scale process sets and complex scenarios.

Enhance the reporting module to generate comprehensive scheduling performance reports.

Implement a data storage system to track user analysis and comparison history.

REFERENCES

Abraham Silberschatz, Peter B. Galvin, Greg Gagne Operating System Concepts (10th Edition),

Wiley Publications.

Andrew S. Tanenbaum Modern Operating Systems, Pearson Education.

William Stallings

Operating Systems: Internals and Design Principles.

GeeksforGeeks – CPU Scheduling

TutorialsPoint – Operating System Scheduling